Record of Changes Since Last Repository Push

Session-based Music Queue Reordering System Using Skip Behaviour as Implicit Feedback
William McKenna — MSc Data Science, Birkbeck, University of London

Baseline commit	0b3afa9 "Additional Testing"
Baseline state	Full BCE and PWTS hyperparameter sweeps complete; PWTS parameters optimised
Repository	github.com/willmckenna15/Session-based-Music-queue-Reordering-System-...
Generated	31 July 2026
Scope	All uncommitted working-tree changes against the baseline commit

1. Summary

The baseline commit represents the project after the first complete hyperparameter sweeps. Diagnostic work carried out since then established that the reported validation AUC at that point was not comparable across models, and that three defects were materially affecting results. The changes below address the architecture, the evaluation protocol and the data pipeline. They are grouped by theme rather than by file.

A. Model architecture — in-session feedback

The model previously received no information about what the listener had done earlier in the session. A three-state skip-status input channel (completed / skipped / unknown) was added, carrying the observed outcome of already-played tracks while every unplayed track is marked "unknown". The training loss is now computed on query positions only, because a context position would otherwise read its own label from its input. Measured effect: per-session AUC rose from 0.578 to approximately 0.607.

B. Model architecture — context/query attention mask

Attention was purely causal, so each candidate track was scored conditional on the tracks preceding it in the realised queue — the very ordering the system exists to change. A block mask was introduced: context positions remain causal among themselves, while query positions attend to the whole context and to themselves but never to one another. Each candidate is now scored independently of queue order. Measured effect on AUC: neutral, which means the more defensible formulation was obtained at no cost in performance.

C. Evaluation protocol

Evaluation previously sampled a single random context/query split per session and discarded the session whenever that draw failed a filter. This was replaced with multi-point evaluation, which enumerates every valid split point, samples up to five evenly, and averages within a session before averaging across sessions. Session coverage rises from roughly 46 per cent to roughly 79 per cent of the validation set, and run-to-run variation from window sampling is eliminated. NDCG@5 was added alongside per-session AUC.

D. Evaluation protocol — random number generator

The evaluation function reset the global PyTorch seed on every call. Because it ran after each training epoch, this restored the generator to an identical state, making dropout masks and batch shuffling byte-identical from epoch two onward. Seeding was moved to a local generator.

E. Data pipeline ordering

Session validity filtering and sequential feature derivation both ran before the audio-feature join, which subsequently removed 9.1 per cent of rows. This left sessions below the stated minimum length and features referring to tracks no longer present: 76.4 per cent of sessions had a stale song_pos, and 11.2 per cent of rows sat at a sequence discontinuity. The pipeline was restructured so that sessions are constructed, joined to audio features, and only then filtered and given sequential features. A new module, feature_and_filter.py, performs the post-join stage.

F. Feature representation

Audio features are now standardised per session rather than globally, expressing each track relative to the queue it appears in. The feature lists in SASRec.py and SASRec_tuning.py were aligned so that hyperparameters are selected on the same inputs the final model trains on, and actively_selected was removed from the model feature set on deployability grounds: it encodes the previous track's outcome, which is unknowable for an unplayed queue track.

2. Files Changed

File	Status	Nature of change
`Data Scripts/feature_and_filter.py`	NEW	Post-join filtering and sequential feature derivation
`Data Scripts/session_compiler.py`	Modified	Reduced to session construction only; returns a DataFrame
`Data Scripts/audio_features_clean.py`	Modified	Global standardisation removed; input path corrected
`Data Scripts/dataset_splitter.py`	Modified	Per-session standardisation added; reads post-filter parquet
`Data Scripts/data_processing_main.py`	Modified	New pipeline stage; per-file deletion guards
`Model Scripts/SASRec_lib.py`	Modified	Status channel, block mask, multi-point evaluation, NDCG@5
`Model Scripts/SASRec.py`	Modified	Feature list aligned; updated call signatures
`Model Scripts/SASRec_tuning.py`	Modified	Feature list aligned; updated call signatures
`Model Scripts/Loss_functions.py`	Modified	PWTS time-weight formulation
`Model Scripts/model_test.py`	Modified	Signature updates
`Model Scripts/Log_regression.py`	Modified	Extended baseline feature set
`.gitignore`	Modified	Working-notes exclusion
`Models/*.csv, Models/*.pt`	Deleted	Sweep results and checkpoints from the superseded architecture

Deleted result files were produced by the superseded architecture and evaluation protocol and are no longer comparable with current output. They should be regenerated before any figure is reported.

3. New File: Data Scripts/feature_and_filter.py

This module performs every step that must happen after the audio-feature join: it derives the skip label, resolves track lengths (which removes rows with unresolvable identifiers), applies the session validity filter, and only then computes the sequential features that depend on adjacency.

```
import pandas as pd
from datetime import timedelta

MIN_SONGS = 7


def filter_valid_sessions(df, min_songs=MIN_SONGS):
    """Same three criteria as before, applied to the dataframe rather than to lists of dicts.

    It has to work on the dataframe now because the filter runs after the audio join, where the
    data is already tabular. Re-run it after any step that drops rows, or the length guarantee
    silently stops holding.
    """
    g = df.groupby("session_id")
    keep = (
        (g["session_id"].transform("size") >= min_songs) &
        (g["Artist Name"].transform("nunique") > 1) &
        (g["skipped"].transform("max") > 0)
    )
    return df[keep]

def amend_reason_start(df):
    reason_start_index = {
        "fwdbtn" : 0,
        "trackdone" : 1,
        "clickrow" : 2,
        "backbtn" : 3,
        "playbtn" : 4
    }

    df["reason_start"] = df["reason_start"].map(reason_start_index).fillna(5)
    return df



def historical_skip_rate(df):

    df = df.sort_values(['user_id', 'spotify_track_uri', 'ts'])


    grouped = df.groupby(['user_id', 'spotify_track_uri'])['skipped']

    cumulative_skips = grouped.transform(lambda x: x.shift(1).expanding().sum())
    cumulative_plays = grouped.transform(lambda x: x.shift(1).expanding().count())

    df['historical_skip_rate'] = (cumulative_skips / cumulative_plays).fillna(0.0)

    df = df.sort_values(['user_id', 'session_id', 'ts'])

    return df

def historical_artist_skip_rate(df):

    df = df.sort_values(['user_id', 'Artist Name', 'ts'])

    grouped = df.groupby(['user_id', 'Artist Name'])['skipped']

    cumulative_skips = grouped.transform(lambda x: x.shift(1).expanding().sum())
    cumulative_plays = grouped.transform(lambda x: x.shift(1).expanding().count())

    df['historical_artist_skip_rate'] = (cumulative_skips / cumulative_plays).fillna(0.0)

    df = df.sort_values(['user_id', 'session_id', 'ts'])

    return df

def add_behavioural_features(df):
    df = df.sort_values(['session_id', 'ts'])
    df['shuffle'] = df['shuffle'].astype(int)
    df['is_repeat_track'] = (df.groupby(['session_id', 'spotify_track_uri']).cumcount() > 0).astype(int)
    df['same_artist_as_prev'] = (
        df.groupby('session_id')['Artist Name'].shift(1) == df['Artist Name']
    ).fillna(False).astype(int)
    return df

def song_position(df):
    df = df.sort_values(['session_id', 'ts'])
    df['song_pos'] = df.groupby('session_id').cumcount()
    return df

def add_track_length(df):
    completed = df[df['reason_end'] == 'trackdone'].groupby('spotify_track_uri')['ms_played'].max()

    all_max = df.groupby('spotify_track_uri')['ms_played'].max()
    track_lengths = all_max.copy()
```

```
    track_lengths = track_lengths.clip(lower=240000) #4 minutes
    track_lengths.update(completed)
    df['track_length'] = df['spotify_track_uri'].map(track_lengths)
    df['ms_played'] = df[['ms_played', 'track_length']].min(axis=1)
    df = df[df['track_length'] > 0]
    return df

def feature_vectors(sessions):
    sessions = sessions.sort_values(["session_id", "ts"]).reset_index(drop=True)
    sessions["ts"] = pd.to_datetime(sessions["ts"])
    sessions["hour"] = sessions["ts"].dt.hour
    sessions["day_of_week"] = sessions["ts"].dt.dayofweek
    sessions["skipped"] = sessions["reason_end"].isin(["fwdbtn", "clickrow"]).astype(int)
    sessions["actively_selected"] = (sessions["reason_start"] == "clickrow").astype(int)

    return sessions

def main():
    print("Reading merged sessions...")
    df = pd.read_parquet("../RAW Data/Combined_Streaming_History.parquet")
    before_rows, before_sessions = len(df), df["session_id"].nunique()

    # Order matters. Everything that DROPS rows runs first, then the validity filter, then the
    # features that depend on sequential adjacency. Deriving song_pos or same_artist_as_prev
    # before the drops would leave them referring to tracks no longer in the session.
    print("Creating feature vectors...")
    Filtered_sessions = feature_vectors(df)
    Filtered_sessions = add_track_length(Filtered_sessions)      # drops track_length <= 0 / NaN

    print("Applying Secondary Filters...")
    Filtered_sessions = filter_valid_sessions(Filtered_sessions)
    print(f"  {before_rows:,} rows / {before_sessions:,} sessions -> "
          f"{len(Filtered_sessions):,} rows / {Filtered_sessions['session_id'].nunique():,} sessions")

    print("Deriving sequential features...")
    Filtered_sessions = historical_skip_rate(Filtered_sessions)
    Filtered_sessions = historical_artist_skip_rate(Filtered_sessions)
    Filtered_sessions = add_behavioural_features(Filtered_sessions)
    Filtered_sessions = song_position(Filtered_sessions)
    Filtered_sessions = amend_reason_start(Filtered_sessions)

    lengths = Filtered_sessions.groupby("session_id").size()
    print(f"  session length: min {lengths.min()}, median {lengths.median():.0f}, max {lengths.max()}")
    if lengths.min() < MIN_SONGS:
        print(f"  WARNING: {(lengths < MIN_SONGS).sum()} session(s) below the {MIN_SONGS}-song floor")

    print("Writing to parquet...")
    Filtered_sessions.to_parquet("../RAW Data/Filtered_Sessions.parquet", index=False)
    print("Filtered sessions saved")

    valid_sessions = Filtered_sessions["session_id"].nunique()
    song_count = len(Filtered_sessions)
    agg_skip_count = int(Filtered_sessions["skipped"].sum())

    print("Formatting Statistics...")
    if not Filtered_sessions.empty:
        user_stats = Filtered_sessions.groupby("user_id").agg(
            valid_sessions=("session_id", "nunique"),
            avg_songs_per_session=("session_id", lambda x: len(x) / x.nunique()),
            skip_rate=("reason_end", lambda x: x.isin(["fwdbtn", "clickrow"]).mean() * 100)
        ).reset_index().rename(columns={
            "user_id": "UUID",
            "valid_sessions": "Number of Sessions",
            "avg_songs_per_session": "Average Songs per Session",
            "skip_rate": "Skip Rate"
        })

        session_file = "../Session Data.csv"

        try:
            sessions_df = pd.read_csv(session_file)
        except (FileNotFoundError, pd.errors.EmptyDataError):
            sessions_df = pd.DataFrame(columns=["UUID", "Number of Sessions", "Average Songs per Session", "Skip Rate"])

        print("writing to csv..")
        sessions_df = pd.concat([sessions_df, user_stats], ignore_index=True)
        sessions_df.to_csv(session_file, index=False)
        print("csv saved")

        print(user_stats)
        print(f"\nTotal valid sessions: {valid_sessions}")
        print(f"\nNumber of songs: {song_count}")
        print(f"\nBaseline Skip Rate: {agg_skip_count / song_count * 100:.2f}%")
```

4. Full Diff Against Baseline

Unified diff of all source files against commit 0b3afa9. Lines beginning with a minus sign were removed; lines beginning with a plus sign were added.

.gitignore

```
@@ -8,6 +8,9 @@ Volunteer Data
 ~$Data Collection.xlsx


+# Working notes (not for submission)
+Info/AUC Diagnosis and Action Plan.md
+
 # Python
 __pycache__/
 *.pyc
```

Data Scripts/audio_features_clean.py

```
@@ -1,10 +1,10 @@
 import pandas as pd
-from sklearn.preprocessing import StandardScaler
+import numpy as np
 import glob

 def main():
     print("Reading parquet...")
-    parquet_path = '../RAW Data/Filtered_sessions.parquet'
+    parquet_path = '../RAW Data/Streaming_Sessions.parquet'
     Streaming_history = pd.read_parquet(parquet_path)
     print("Parquet Read")

@@ -57,12 +57,6 @@ def main():

     print(" ")

-    print("Normalising Dataset")
-    scaler = StandardScaler()
-    cols_to_normalise = ['danceability', 'energy', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', '
-    Streaming_history_merged[cols_to_normalise] = scaler.fit_transform(Streaming_history_merged[cols_to_normalise])
-    print("Data Normalised")
-
     print("Writing to csv...")
     Streaming_history_merged.to_parquet('../RAW Data/Combined_Streaming_History.parquet', index=False)
     print("Combined Streaming History saved")
```

Data Scripts/data_processing_main.py

```
@@ -4,29 +4,29 @@ import session_compiler
 import dataset_splitter
 import os
 import glob
+import feature_and_filter

 print("removing pre-existing files...")
-try:
-    files = glob.glob("../RAW Data/Streaming_history_*.csv")
-    for f in files:
+# Each deletion is guarded separately. Previously one try/except wrapped them all, so a single
+# already-missing file aborted the rest and left stale parquets behind for the next stage to
+# read.
+stale = (glob.glob("../RAW Data/Streaming_history_*.csv")
+         + glob.glob("../RAW Data/*.parquet")
+         + ["../RAW Data/Combined_Streaming_History.csv", "../Session Data.csv"])
+for f in stale:
+    try:
         os.remove(f)
-
-    os.remove("../RAW Data/Combined_Streaming_History.csv")
-    os.remove("../Session Data.csv")
-    p_files=glob.glob("../RAW Data/*.parquet")
-    for p in p_files:
-        os.remove(p)
-except FileNotFoundError:
-    pass
+    except FileNotFoundError:
+        pass
 print("Pre-existing files removed")

-print("\033[1;4mCombining RAW Datasets\033[0m")
+print("\033[1;4m\nCombining RAW Datasets\033[0m")
 Project_Json2csv.main()
 print("\033[1;4mRAW Datasets Combined\033[0m")

 print(" ")
```

```
-print("\033[1;4mConstructing Streaming Sessions\033[0m")
+print("\033[1;4m\nConstructing Streaming Sessions\033[0m")
 session_compiler.main()
 print("\033[1;4mStreaming Sessions Finalised\033[0m")

@@ -36,6 +36,10 @@ print("\033[1;4mMerging Streaming Sessions with Audio Features \033[0m")
 audio_features_clean.main()
 print("\033[1;4mDatasets Merged\033[0m")

-print("\033[1;4mSplitting Dataset into training, validation & testing...\033[0m")
+print("\033[1;4m\nCreating sequential features and applying filters... \033[0m")
+feature_and_filter.main()
+print("\033[1;4mFilters applied\033[0m")
+
+print("\033[1;4m\nSplitting Dataset into training, validation & testing...\033[0m")
 dataset_splitter.main()
 print("\033[1;4mDatasets split.\033[0m")
\ No newline at end of file
```

Data Scripts/dataset_splitter.py

```
@@ -1,11 +1,23 @@
+import numpy as np
 import pandas as pd


 def main():
-    INPUT = '../RAW Data/Combined_Streaming_History.parquet'
+    INPUT = '../RAW Data/Filtered_Sessions.parquet'

     sessions = pd.read_parquet(INPUT)

+    print("Normalising Dataset (per user)")
+    cols_to_normalise = ['danceability', 'energy', 'speechiness', 'acousticness',
+                         'instrumentalness', 'liveness', 'valence', 'loudness', 'tempo']
+    Streaming_sessions  = sessions.groupby('session_id')[cols_to_normalise]
+    mu = Streaming_sessions.transform('mean')
+    sd = Streaming_sessions.transform('std').replace(0, np.nan)
+    sessions[cols_to_normalise] = (
+        (sessions[cols_to_normalise] - mu) / sd
+    ).fillna(0.0)
+    print("Data Normalised")
+
     users = sessions["user_id"].unique()

     training_parts = []
```

Data Scripts/session_compiler.py

```
@@ -4,7 +4,7 @@ from datetime import timedelta

 def build_sessions(df):
     print("Constructing sessions...")
-
+
     df = df.copy()
     df["ts"] = pd.to_datetime(df["ts"])
     df = df.sort_values(["user_id", "ts"]).reset_index(drop=True)
@@ -16,142 +16,26 @@ def build_sessions(df):
         (df["user_id"] != df["user_id"].shift(1))
     )

-    df["session_id"] = new_session.cumsum()
+    # session_id stays "<user_id>_<n>" so it remains unique per user and readable downstream
+    df["session_id"] = df["user_id"].astype(str) + "_" + new_session.cumsum().astype(str)
     df = df.drop(columns=["time_gap"])

-    streaming_sessions = {
-        session_id: group.to_dict('records')
-        for session_id, group in df.groupby("session_id")
-    }
-
     print("All Sessions Constructed")
-    return streaming_sessions
+    return df

-def is_valid_session(songs):
-    if len(songs) < 10:
-        return False
-    if len(set(song["Artist Name"] for song in songs)) == 1:
-        return False
-    if not any(song["reason_end"] in ("fwdbtn", "clickrow") for song in songs):
-        return False
-    return True

 def session_compiler():
     df = pd.read_csv("../RAW Data/Combined_Streaming_History.csv")
-
-
-    streaming_sessions = build_sessions(df)
-
```

```
-    print("Applying Secondary Filters...")
-    valid = {k: v for k, v in streaming_sessions.items() if is_valid_session(v)}
-
-    Filtered_sessions = pd.DataFrame([
-        {**song, "session_id": f"{song['user_id']}_{session_no}"}
-        for session_no, songs in valid.items()
-        for song in songs
-    ])
-
-    if Filtered_sessions.empty:
-        print("No valid sessions found.")
-        return Filtered_sessions, 0, 0, 0
-
-    valid_sessions = Filtered_sessions["session_id"].nunique()
-    song_count = len(Filtered_sessions)
-    agg_skip_count = Filtered_sessions["reason_end"].isin(["fwdbtn", "clickrow"]).sum()
-    print("Sessions Filtered")
-    return Filtered_sessions, valid_sessions, song_count, agg_skip_count
-
-def feature_vectors(sessions):
-    sessions = sessions.sort_values(["session_id", "ts"]).reset_index(drop=True)
-    sessions["ts"] = pd.to_datetime(sessions["ts"])
-    sessions["hour"] = sessions["ts"].dt.hour
-    sessions["day_of_week"] = sessions["ts"].dt.dayofweek
-    sessions["skipped"] = sessions["reason_end"].isin(["fwdbtn", "clickrow"]).astype(int)
-    sessions["actively_selected"] = (sessions["reason_start"] == "clickrow").astype(int)
-
-    return sessions
-
-def historical_skip_rate(df):
-
-    df = df.sort_values(['user_id', 'spotify_track_uri', 'session_id'])
-
-
-    grouped = df.groupby(['user_id', 'spotify_track_uri'])['skipped']
-
-    cumulative_skips = grouped.transform(lambda x: x.shift(1).expanding().sum())
-    cumulative_plays = grouped.transform(lambda x: x.shift(1).expanding().count())
-
-    df['historical_skip_rate'] = (cumulative_skips / cumulative_plays).fillna(0.0)
-
-    df = df.sort_values(['user_id', 'session_id', 'ts'])
-
-    return df
-
-def song_position(df):
-    df = df.sort_values(['session_id', 'ts'])
-    df['song_pos'] = df.groupby('session_id').cumcount()
-    return df
-
-def add_track_length(df):
-    completed = df[df['reason_end'] == 'trackdone'].groupby('spotify_track_uri')['ms_played'].max()
-
-    all_max = df.groupby('spotify_track_uri')['ms_played'].max()
-
-    track_lengths = all_max.copy()
-    track_lengths = track_lengths.clip(lower=240000) #4 minutes
-    track_lengths.update(completed)
-    df['track_length'] = df['spotify_track_uri'].map(track_lengths)
-    df['ms_played'] = df[['ms_played', 'track_length']].min(axis=1)
-    df = df[df['track_length'] > 0]
-    return df
-
-
-
-
+    return build_sessions(df)


 def main():
-    Filtered_sessions, valid_sessions, song_count, agg_skip_count = session_compiler()
-
-    print("Creating features vectors...")
-    Filtered_sessions = feature_vectors(Filtered_sessions)
-    Filtered_sessions = historical_skip_rate(Filtered_sessions)
-    Filtered_sessions = song_position(Filtered_sessions)
-    Filtered_sessions = add_track_length(Filtered_sessions)
-    print("Writing to parquet...")
-    print(Filtered_sessions.head(5))
-
-    Filtered_sessions.to_parquet("../RAW Data/Filtered_Sessions.parquet", index=False)
-    print("Filtered sessions saved")
-
-    print("Formatting Statistics...")
-    if not Filtered_sessions.empty:
-        user_stats = Filtered_sessions.groupby("user_id").agg(
-            valid_sessions=("session_id", "nunique"),
-            avg_songs_per_session=("session_id", lambda x: len(x) / x.nunique()),
-            skip_rate=("reason_end", lambda x: x.isin(["fwdbtn", "clickrow"]).mean() * 100)
-        ).reset_index().rename(columns={
-            "user_id": "UUID",
-            "valid_sessions": "Number of Sessions",
-            "avg_songs_per_session": "Average Songs per Session",
-            "skip_rate": "Skip Rate"
```

```
-        })
-
-        session_file = "../Session Data.csv"
-
-        try:
-            sessions_df = pd.read_csv(session_file)
-        except (FileNotFoundError, pd.errors.EmptyDataError):
-            sessions_df = pd.DataFrame(columns=["UUID", "Number of Sessions", "Average Songs per Session", "Skip Rate"])
-
-        print("writing to csv..")
-        sessions_df = pd.concat([sessions_df, user_stats], ignore_index=True)
-        sessions_df.to_csv(session_file, index=False)
-        print("csv saved")
-
-        print(user_stats)
-        print(f"\nTotal valid sessions: {valid_sessions}")
-        print(f"\nNumber of songs: {song_count}")
-        print(f"\nBaseline Skip Rate: {agg_skip_count / song_count * 100:.2f}%")
-
-
+    # Sessions are only CONSTRUCTED here. Validity filtering and feature derivation happen in
+    # feature_and_filter.py, after the audio join, because that join drops rows and would
+    # otherwise leave sessions that no longer meet the criteria and features computed over
+    # tracks that are no longer present.
+    streaming_sessions = session_compiler()
+
+    streaming_sessions.to_parquet("../RAW Data/Streaming_Sessions.parquet", index=False)
+    print(f"Streaming sessions saved: {len(streaming_sessions):,} rows, "
+          f"{streaming_sessions['session_id'].nunique():,} sessions")
```

Model Scripts/Log_regression.py

```
@@ -10,8 +10,10 @@ extra_cols = ['user_id', 'spotify_track_uri']



-features = ['tempo','mode', 'danceability', 'energy', 'loudness','speechiness', 'acousticness', 'instrumentalness', 'liveness'
-
+features = ['tempo', 'mode', 'danceability', 'energy', 'loudness', 'speechiness',
+            'acousticness', 'instrumentalness', 'liveness', 'valence',
+            'hour', 'day_of_week', 'actively_selected', 'historical_skip_rate',
+            'historical_artist_skip_rate', 'shuffle', 'is_repeat_track', 'same_artist_as_prev']
 target = 'skipped'

 training_df = pd.read_parquet(training_file, columns=features + [target] + extra_cols)
```

Model Scripts/Loss_functions.py

```
@@ -47,19 +47,15 @@ class PWTSLoss(nn.Module):
     def forward(self, preds, labels, song_pos, lengths, ms_played, track_length, historical_skip_rate):
         percentage_listen = ms_played / torch.clamp(track_length, min=1e-8)
         percentage_pos = song_pos / torch.clamp(lengths, min=1e-8)
-
         position_weight = 1 + torch.exp(-percentage_pos * 3)
-        time_weight = 1 + torch.exp(-percentage_listen * 30)
-        '''
-        linear_part = 2 - (2 - 1.0) * (percentage_listen / 0.15)
+        linear_part = 2 - (2 - 1.0) * (percentage_listen / 0.1)
         time_weight = torch.where(
-            percentage_listen <= 0.15,
+            percentage_listen <= 0.1,
             torch.clamp(linear_part, min=1.0),
             torch.tensor(1.0, device=percentage_listen.device))
-        '''
         historical_weight = 1 + torch.sigmoid((historical_skip_rate - 0.5) * 6)

-        combined_weight = position_weight * time_weight * historical_weight
+        combined_weight = time_weight * historical_weight * position_weight
         combined_weight = combined_weight/combined_weight.mean()
         per_sample_loss = self.bce(preds, labels)
         return (combined_weight * per_sample_loss).mean()
```

Model Scripts/SASRec.py

```
@@ -13,7 +13,8 @@ from Loss_functions import get_loss_criterion, parse_args, DrRLLoss, PWTSLoss

 features = ['tempo', 'mode', 'danceability', 'energy', 'loudness', 'speechiness',
             'acousticness', 'instrumentalness', 'liveness', 'valence',
-            'hour', 'day_of_week', 'actively_selected']
+            'hour', 'day_of_week', 'historical_skip_rate',
+            'historical_artist_skip_rate', 'shuffle', 'is_repeat_track', 'same_artist_as_prev']
 target = 'skipped'

 device = torch.device("mps" if torch.mps.is_available() else "cpu")
@@ -67,7 +68,7 @@ def run_training(seed=None, verbose=True):
         np.random.seed(seed)
         random.seed(seed)

-    model = SASRec(feature_no=len(features), args=args_mod).to(device)
```

```
+    model = SASRec(feature_no=len(features), num_reason_classes=6, args=args_mod).to(device)
     criterion = build_criterion()
     optimizer = torch.optim.Adam(
         list(model.parameters()) + list(criterion.parameters()),
@@ -79,7 +80,7 @@ def run_training(seed=None, verbose=True):

     for epoch in range(100):
         train_loss = train_epoch(model, train_loader, optimizer, criterion, device, loss_name=loss_name)
-        val_auc = evaluate_sequential(model, val_loader, device)
+        val_auc, _ = evaluate_sequential(model, val_loader, device)

         if verbose:
             tqdm.write(f"Epoch {epoch+1} | Loss: {train_loss:.4f} | Val AUC: {val_auc:.4f}")
```

Model Scripts/SASRec_lib.py

```
@@ -32,9 +32,10 @@ def handle_interrupt(sig, frame):
         csv_path = f'../Models/sasrec_grid_search_{loss_name}_results.csv'
         if os.path.exists(csv_path):
             os.remove(csv_path)
-            sys.stdout.write("Progress reset. Rerun the script to start fresh.\n")
-            sys.stdout.flush()
-        os._exit(0)
+            print(f"Progress reset ({csv_path} deleted). Rerun the script to start fresh.")
+        else:
+            print(f"Nothing to reset - {csv_path} does not exist.")
+        exit(0)
     else:
         sys.stdout.write("Invalid choice, resuming run...\n")
         sys.stdout.flush()
@@ -51,6 +52,7 @@ class SessionDataset(Dataset):
         self.ms_played = []
         self.track_length = []
         self.historical_skip_rate = []
+        self.reason_start = []

         for session_id, session in df.groupby("session_id"):
             session = session.sort_values("ts")
@@ -62,6 +64,7 @@ class SessionDataset(Dataset):
             self.ms_played.append(torch.tensor(session['ms_played'].values, dtype=torch.float32))
             self.track_length.append(torch.tensor(session['track_length'].values, dtype=torch.float32))
             self.historical_skip_rate.append(torch.tensor(session['historical_skip_rate'].values, dtype=torch.float32))
+            self.reason_start.append(torch.tensor(session['reason_start'].values, dtype=torch.long))

     def __len__(self):
         return len(self.sessions)
@@ -69,19 +72,20 @@ class SessionDataset(Dataset):
     def __getitem__(self, idx):
         return (self.sessions[idx], self.labels[idx], self.song_pos[idx],
                 self.ms_played[idx], self.track_length[idx],
-                self.historical_skip_rate[idx])
+                self.historical_skip_rate[idx], self.reason_start[idx])


 def collate_fn(batch):
-    sessions, labels, song_pos, ms_played, track_length, historical_skip_rate = zip(*batch)
+    sessions, labels, song_pos, ms_played, track_length, historical_skip_rate, reason_start = zip(*batch)
     sessions_padded = nn.utils.rnn.pad_sequence(sessions, batch_first=True)
     labels_padded = nn.utils.rnn.pad_sequence(labels, batch_first=True)
     song_pos_padded = nn.utils.rnn.pad_sequence(song_pos, batch_first=True)
     ms_played_padded = nn.utils.rnn.pad_sequence(ms_played, batch_first=True)
     track_length_padded = nn.utils.rnn.pad_sequence(track_length, batch_first=True)
     historical_skip_rate_padded = nn.utils.rnn.pad_sequence(historical_skip_rate, batch_first=True)
+    reason_start_padded = nn.utils.rnn.pad_sequence(reason_start, batch_first=True, padding_value=-100)
     lengths = torch.tensor([len(s) for s in sessions])
-    return sessions_padded, labels_padded,song_pos_padded, ms_played_padded, track_length_padded,historical_skip_rate_padded,
+    return sessions_padded, labels_padded,song_pos_padded, ms_played_padded, track_length_padded,historical_skip_rate_padded,

 ##Model

@@ -99,15 +103,33 @@ class PointWiseFeedForward(torch.nn.Module):
         outputs = outputs.transpose(-1, -2)
         return outputs

+MIN_CONTEXT = 3
+MIN_WINDOW_LENGTH = 5
+
+def build_attn_mask(seq_len, boundaries, num_heads, device):
+    """True = blocked. attn_mask[i, j] blocks query position i from attending key j."""
+    idx    = torch.arange(seq_len, device=device)
+    causal = idx.unsqueeze(1) < idx.unsqueeze(0)              # (L,L)  block j > i
+    is_q   = idx.unsqueeze(0) >= boundaries.unsqueeze(1).to(device)      # (N,L)  query positions
+    qq     = is_q.unsqueeze(2) & is_q.unsqueeze(1)            # (N,L,L) query attending query
+    eye    = torch.eye(seq_len, dtype=torch.bool, device=device)
+    mask   = causal.unsqueeze(0) | (qq & ~eye)                # keep the diagonal (self)
+    return mask.repeat_interleave(num_heads, dim=0)           # (N*heads, L, L)
+
+

 class SASRec(torch.nn.Module):
-    def __init__(self, feature_no, args):
+
+    def __init__(self, feature_no, num_reason_classes, args):
```

```
         super(SASRec, self).__init__()
         self.feature_no = feature_no
         self.dev = args.device
         self.norm_first = args.norm_first
+        self.num_heads = args.num_heads
         self.feature_proj = torch.nn.Linear(self.feature_no, args.hidden_units)
         self.pos_emb = torch.nn.Embedding(args.maxlen + 1, args.hidden_units, padding_idx=0)
+        self.status_emb = nn.Embedding(3, args.hidden_units)
+        nn.init.zeros_(self.status_emb.weight[2])
         self.emb_dropout = torch.nn.Dropout(p=args.dropout_rate)
         self.attention_layernorms = torch.nn.ModuleList()
         self.attention_layers = torch.nn.ModuleList()
@@ -120,38 +142,38 @@ class SASRec(torch.nn.Module):
             self.forward_layernorms.append(torch.nn.LayerNorm(args.hidden_units, eps=1e-8))
             self.forward_layers.append(PointWiseFeedForward(args.hidden_units, args.dropout_rate))
         self.output_layer = torch.nn.Linear(args.hidden_units, 1)
+        self.aux_head = nn.Linear(args.hidden_units, num_reason_classes)
         self.sigmoid = torch.nn.Sigmoid()

-    def seq2feats(self, x, lengths):
-        seqs = self.feature_proj(x)
+    def seq2feats(self, x, lengths, status, boundaries):
         N, seq_len, _ = x.shape
         poss = torch.arange(1, seq_len + 1, device=self.dev).unsqueeze(0).expand(N, -1).clone()
         poss = poss.clamp(max=self.pos_emb.num_embeddings - 1)
         for i, l in enumerate(lengths):
             poss[i, l:] = 0
-        seqs += self.pos_emb(poss)
+        seqs = self.feature_proj(x) + self.status_emb(status) + self.pos_emb(poss)
         seqs = self.emb_dropout(seqs)
         tl = seq_len
-        causal_mask = ~torch.tril(torch.ones((tl, tl), dtype=torch.bool, device=self.dev))
+        attn_mask = build_attn_mask(tl, boundaries, self.num_heads, self.dev)
         for i in range(len(self.attention_layers)):
             seqs = torch.transpose(seqs, 0, 1)
             if self.norm_first:
                 x_ = self.attention_layernorms[i](seqs)
-                mha_out, _ = self.attention_layers[i](x_, x_, x_, attn_mask=causal_mask)
+                mha_out, _ = self.attention_layers[i](x_, x_, x_, attn_mask=attn_mask)
                 seqs = seqs + mha_out
                 seqs = torch.transpose(seqs, 0, 1)
                 seqs = seqs + self.forward_layers[i](self.forward_layernorms[i](seqs))
             else:
-                mha_out, _ = self.attention_layers[i](seqs, seqs, seqs, attn_mask=causal_mask)
+                mha_out, _ = self.attention_layers[i](seqs, seqs, seqs, attn_mask=attn_mask)
                 seqs = self.attention_layernorms[i](seqs + mha_out)
                 seqs = torch.transpose(seqs, 0, 1)
                 seqs = self.forward_layernorms[i](seqs + self.forward_layers[i](seqs))
         return self.last_layernorm(seqs)

-    def forward(self, x, lengths):
-        feats = self.seq2feats(x, lengths)
+    def forward(self, x, lengths, status, boundaries):
+        feats = self.seq2feats(x, lengths, status, boundaries)
         skip_probs = self.sigmoid(self.output_layer(feats)).squeeze(-1)
-        return skip_probs
+        return skip_probs, self.aux_head(feats)

 def log_test(mean_auc, std_auc, experiment_name, config_description):
     log_path = '../Models/test_log.csv'
@@ -173,12 +195,13 @@ def log_test(mean_auc, std_auc, experiment_name, config_description):

 ##Training

-def train_epoch(model, loader, optimizer, criterion, device, loss_name='bce'):
+def train_epoch(model, loader, optimizer, criterion, device, loss_name='bce', aux_weight=0.3):
     model.train()
     total_loss = 0
     progress = tqdm(loader, desc="Training", leave=False)
-    for sessions, labels, song_pos, ms_played, track_length, historical_skip_rate, lengths in progress:
-        sessions, labels = sessions.to(device), labels.to(device)
+    aux_criterion = nn.CrossEntropyLoss(ignore_index=-100)
+    for sessions, labels, song_pos, ms_played, track_length, historical_skip_rate, reason_start, lengths in progress:
+        sessions, labels, reason_start = sessions.to(device), labels.to(device), reason_start.to(device)
         if loss_name == 'pwts':
             lengths_expanded = torch.zeros_like(labels)
             for i, length in enumerate(lengths):
@@ -188,12 +211,20 @@ def train_epoch(model, loader, optimizer, criterion, device, loss_name='bce'):
             track_length = track_length.to(device)
             lengths_expanded = lengths_expanded.to(device)
             historical_skip_rate = historical_skip_rate.to(device)
+        status = torch.full_like(labels, 2, dtype=torch.long)
+        boundaries = torch.empty(len(lengths), dtype=torch.long)
+        for i, L in enumerate(lengths):
+            L = int(L)
+            b = torch.randint(MIN_CONTEXT, max(int(L), MIN_CONTEXT + 1), (1,)).item()
+            b=min(b,L)
+            boundaries[i] = b
+            status[i, :b] = labels[i, :b].long()

         optimizer.zero_grad()
-        preds = model(sessions, lengths)
+        preds, aux_logits = model(sessions, lengths, status, boundaries)
         mask = torch.zeros_like(labels, dtype=torch.bool)
```

```
         for i, length in enumerate(lengths):
-            mask[i, :length] = True
+            mask[i, boundaries[i]:int(length)] = True

         if loss_name == "pwts":
             loss = criterion(preds[mask], labels[mask], song_pos[mask], lengths_expanded[mask], ms_played[mask], track_length[
@@ -203,32 +234,95 @@ def train_epoch(model, loader, optimizer, criterion, device, loss_name='bce'):
         else:
             loss = criterion(preds[mask], labels[mask])

-        loss.backward()
+
+        aux_target = reason_start.clone()
+        aux_target[~mask] = -100
+        aux_loss = aux_criterion(aux_logits.reshape(-1, aux_logits.size(-1)),
+                                 aux_target.reshape(-1))
+
+        combined_loss = loss + aux_weight * aux_loss
+
+        combined_loss.backward()
         optimizer.step()
-        total_loss += loss.item()
-        progress.set_postfix(loss=f"{loss.item():.4f}")
+        total_loss += combined_loss.item()
+        progress.set_postfix(loss=f"{combined_loss.item():.4f}")
     return total_loss / len(loader)


-def evaluate_sequential(model, loader, device, min_context=5, min_skips_in_context=1, seed=42):
-    torch.manual_seed(seed)
-    np.random.seed(seed)
-    random.seed(seed)
+def valid_splits(y, length, min_context=MIN_CONTEXT, min_skips=1, min_window=MIN_WINDOW_LENGTH):
+    """Every context length at which this session can legitimately be scored."""
+    return [c for c in range(min_context, length - min_window + 1)
+            if y[:c].sum() >= min_skips and len(np.unique(y[c:length])) >= 2]
+
+POINTS_PER_SESSION = 5
+NDCG_K =5
+
+def ndcg_at_k(y, p, k=NDCG_K):
+    """Queue ordered ascending by predicted skip probability; a track NOT skipped is relevant."""
+    order = np.argsort(p, kind='stable')
+    rel   = 1.0 - y[order]
+    disc  = 1.0 / np.log2(np.arange(2, len(rel) + 2))
+    dcg   = float((rel[:k] * disc[:len(rel[:k])]).sum())
+    ideal = np.sort(1.0 - y)[::-1][:k]
+    idcg  = float((ideal * disc[:len(ideal)]).sum())
+    return dcg / idcg if idcg > 0 else 0.0
+
+def pick(cands, points=POINTS_PER_SESSION):
+    """Evenly spaced subset - deterministic, so there is no seed and no run-to-run variation."""
+    if len(cands) <= points:
+        return cands
+    return [cands[k] for k in np.linspace(0, len(cands) - 1, points).astype(int)]
+
+
+def evaluate_sequential(model, loader, device, min_context=MIN_CONTEXT, min_skips_in_context=1,
+                        min_window_length=MIN_WINDOW_LENGTH, points=POINTS_PER_SESSION,
+                        chunk=32):
     model.eval()
-    all_preds = []
-    all_labels = []
+    session_aucs = []
+    session_ndcgs = []
     with torch.no_grad():
-        for sessions, labels, song_pos, ms_played, track_length, historical_skip_rate, lengths in loader:
-            sessions = sessions.to(device)
-            preds = model(sessions, lengths)
-            for i, length in enumerate(lengths):
-                length = length.item()
-                if length < min_context + 1:
-                    continue
-                context_len = torch.randint(min_context, length, (1,)).item()
-                context_skips = labels[i, :context_len].sum().item()
-                if context_skips < min_skips_in_context:
-                    continue
-                all_preds.extend(preds[i, context_len:length].cpu().numpy())
-                all_labels.extend(labels[i, context_len:length].numpy())
-    return roc_auc_score(all_labels, all_preds)
-\ No newline at end of file
+        for sessions, labels, song_pos, ms_played, track_length, hsr, _, lengths in loader:
+            B = sessions.shape[0]
+
+            # 1. enumerate split points; a session is dropped only if NONE are valid
+            rows = []
+            for i in range(B):
+                length = int(lengths[i])
+                y = labels[i, :length].numpy()
+                for c in pick(valid_splits(y, length, min_context,
+                                           min_skips_in_context, min_window_length), points):
+                    rows.append((i, c))
+            if not rows:
+                continue
```

```
+
+            # 2. expand: one row per (session, split point), each with its own status + boundary
+            idx    = torch.tensor([i for i, _ in rows])
+            bounds = torch.tensor([c for _, c in rows], dtype=torch.long)
+            exp_labels = labels[idx]
+            status = torch.full(exp_labels.shape, 2, dtype=torch.long)
+            for r, (_, c) in enumerate(rows):
+                status[r, :c] = exp_labels[r, :c].long()
+
+            # 3. forward in chunks - the attention mask is (rows x heads x L x L), so an
+            #    un-chunked expanded batch can exhaust memory on long sessions
+            preds = []
+            for s in range(0, len(rows), chunk):
+                sl = slice(s, s + chunk)
+                p, _ = model(sessions[idx[sl]].to(device), lengths[idx[sl]],
+                             status[sl].to(device), bounds[sl])
+                preds.append(p.cpu())
+            preds = torch.cat(preds, dim=0)
+
+            # 4. score each window, then average WITHIN session before averaging across sessions
+            g_auc, g_ndcg = {}, {}
+            for r, (i, c) in enumerate(rows):
+                length = int(lengths[i])
+                y = labels[i, c:length].numpy()
+                p = preds[r, c:length].numpy()
+                g_auc.setdefault(i, []).append(roc_auc_score(y, p))
+                g_ndcg.setdefault(i, []).append(ndcg_at_k(y, p))
+            session_aucs.extend(np.mean(v) for v in g_auc.values())
+            session_ndcgs.extend(np.mean(v) for v in g_ndcg.values())
+
+    return np.mean(session_aucs), np.mean(session_ndcgs), len(session_aucs)
+
```

Model Scripts/SASRec_tuning.py

```
@@ -30,10 +30,11 @@ signal.signal(signal.SIGINT, handle_interrupt)

 features = ['tempo', 'mode', 'danceability', 'energy', 'loudness', 'speechiness',
             'acousticness', 'instrumentalness', 'liveness', 'valence',
-            'hour', 'day_of_week', 'actively_selected']
+            'hour', 'day_of_week', 'historical_skip_rate',
+            'historical_artist_skip_rate', 'shuffle', 'is_repeat_track', 'same_artist_as_prev']
 target = 'skipped'

-device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
+device = torch.device("mps" if torch.mps.is_available() else "cpu")
 print(f"Using device: {device}")

 args = parse_args()
@@ -109,7 +110,7 @@ for combo in combinations:
         norm_first=True
     )

-    model = SASRec(feature_no=len(features), args=args_mod).to(device)
+    model = SASRec(feature_no=len(features), num_reason_classes=6, args=args_mod).to(device)

     if loss_name == "drrl":
         criterion = DrRLLoss(gamma=params["gamma"])
@@ -126,29 +127,33 @@ for combo in combinations:
     )


+    best_ndcg = 0
     best_auc = 0
     patience_counter = 0

     for epoch in range(max_epochs):
         train_loss = train_epoch(model, train_loader, optimizer, criterion, device, loss_name=loss_name)
-        val_auc = evaluate_sequential(model, val_loader, device)
-        tqdm.write(f"Epoch {epoch+1} | Loss: {train_loss:.4f} | Val AUC: {val_auc:.4f}")
-        if val_auc > best_auc:
-            best_auc = val_auc
+        val_auc, val_ndcg, _ = evaluate_sequential(model, val_loader, device)
+        tqdm.write(f"Epoch {epoch+1} | Loss: {train_loss:.4f} | Val AUC: {val_auc:.4f} | Val NDCG: {val_ndcg:.4f}")
+        if val_ndcg > best_ndcg:
+            best_ndcg = val_ndcg
             patience_counter = 0
         else:
             patience_counter += 1
             if patience_counter >= 3:
                 break
+        if val_auc > best_auc:
+            best_auc = val_auc
+
     else:
         ran_to_end = 1
         ran_to_end_count += 1


-    print(f"\nBest Val AUC: {best_auc:.4f}")
-    results.append({**params, 'val_auc': best_auc, 'ran_to_end': ran_to_end})
+    print(f"\nBest Val NDCG: {best_ndcg:.4f}, Best Val AUC: {best_auc:.4f}")
+    results.append({**params,'val_ndcg': best_ndcg, 'val_auc': best_auc, 'ran_to_end': ran_to_end})
```

```
-    pd.DataFrame(results).sort_values('val_auc', ascending=False).to_csv(
+    pd.DataFrame(results).sort_values('val_ndcg', ascending=False).to_csv(
         completed_csv, index=False
     )
     combinations_completed += 1
@@ -156,7 +161,7 @@ for combo in combinations:

 ##Summary

-results_df = pd.DataFrame(results).sort_values('val_auc', ascending=False)
+results_df = pd.DataFrame(results).sort_values('val_ndcg', ascending=False)
 print("\n--- Grid Search Results ---")
 print(results_df.to_string(index=False))
```

Model Scripts/model_test.py

```
@@ -1,3 +1,44 @@
-import pandas as pd
-df = pd.read_parquet('../RAW Data/training_data.parquet')
-print(df['actively_selected'].value_counts(normalize=True))
\ No newline at end of file
+from torch.utils.data import DataLoader
+from SASRec_lib import SessionDataset, collate_fn, evaluate_sequential, SASRec
+import torch
+import numpy as np
+import argparse
+import json
+
+features = ['tempo', 'mode', 'danceability', 'energy', 'loudness', 'speechiness',
+            'acousticness', 'instrumentalness', 'liveness', 'valence',
+            'hour', 'day_of_week', 'actively_selected', 'historical_skip_rate',
+            'historical_artist_skip_rate', 'shuffle', 'is_repeat_track', 'same_artist_as_prev']
+target = 'skipped'
+
+device = torch.device("mps" if torch.mps.is_available() else "cpu")
+print(f"Using device: {device}")
+
+print("Loading datasets...")
+train_dataset = SessionDataset('../RAW Data/training_data.parquet', features, target)
+val_dataset = SessionDataset('../RAW Data/validation_data.parquet', features, target)
+print(f"Train sessions: {len(train_dataset)} | Val sessions: {len(val_dataset)}")
+
+train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, collate_fn=collate_fn)
+val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, collate_fn=collate_fn)
+
+with open('../Models/sasrec_pwts_best_params.json', 'r') as f:
+    best_params = json.load(f)
+
+args_mod = argparse.Namespace(
+    device=device,
+    hidden_units=int(best_params['hidden_units']),
+    maxlen=200,
+    dropout_rate=float(best_params['dropout_rate']),
+    num_blocks=int(best_params['num_blocks']),
+    num_heads=int(best_params['num_heads']),
+    norm_first=True
+)
+
+model = SASRec(feature_no=len(features), args=args_mod).to(device)
+model.load_state_dict(torch.load('../Models/sasrec_bce_best.pt', map_location=device))
+model.eval()
+
+train_loader_eval = DataLoader(train_dataset, batch_size=32, shuffle=False, collate_fn=collate_fn)
+train_session_auc, n = evaluate_sequential(model, train_loader_eval, device, min_window_length=10)
+print(f"Session-wise AUC on TRAINING set: {train_session_auc:.4f} across {n} sessions")
```

5. Status and Outstanding Work

The changes above are implemented and verified to run. The following remain outstanding and are documented separately in the project working notes:

- aux_weight is defined in the grid but never passed to the training function, so it is fixed at its default value.
- Hyperparameter selection currently uses an uninitialised best score and aggressive early stopping, and training-seed variance has not been measured.
- The adapted DrRL loss operates on post-sigmoid probabilities rather than unbounded scores, and its beta parameter receives updates from two optimisers.
- Time features are ordinal and unnormalised, and there is no input normalisation layer.
- model_test.py has not been updated for the new model signature.
- MRR is not yet implemented; NDCG@5 and per-session AUC are.

All quantitative figures cited in this document were measured before the data pipeline was restructured. Both the model results and the baselines require regeneration before they are reported.